AI RESCUE MISSION:

FIND THE BEST PATH USING

HEURISTIC SEARCH

PROJECT REPORT FOR THE COURSE

Foundations of Artificial Intelligence

Submitted by

240701340 – MUKHESH KUMAAR R S

240701809 – YUVARAJA A

III YEAR - B.E

in

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

THANDALAM, CHENNAI 602 105

BONAFIDE CERTIFICATE

Certified that the project titled **"AI RESCUE MISSION: FIND THE BEST PATH USING HEURISTIC SEARCH"** is the bonafide work carried out by **Mukhesh kumaar R S (240701340)** and **Yuvaraja A (240701809)** under my supervision. This project has been completed in partial fulfilment of the requirements of the course Foundations of Artificial Intelligence, Department of Computer Science and Engineering, Rajalakshmi Engineering College.

Certified further that, to the best of my knowledge and belief, the work reported herein is original and does not form part of any other project, report, or work submitted previously for any academic award.

This project contributes toward achieving the following **Sustainable Development Goals (SDGs):**

1. Good Health and Well-being (SDG 3)
2. Industry, Innovation and Infrastructure (SDG 9)
3. Sustainable Cities and Communities (SDG 11)

Submitted for the Practical Examination held on _____

Signature of the Faculty Incharge with date

Signature Examiner-1

Signature Examiner

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION	5
	Objectives and scope	6
2	LITERATURE REVIEW	7
	Practical pathfinding studies	8
	Research gap	9
3	PROPOSED SYSTEM	10
3.1	Architecture diagram	11
4	MODULES DESCRIPTION	12
4.1	Core A* Engine	12
4.2	Heuristic Manhattan Engine	13
4.3	Game / Rescue Engine	14
4.4	Diagnostics & Trace	15
5	IMPLEMENTATION AND RESULTS	16
5.1	Experimental setup	16
5.2	Results & comparison with BFS	17
	Interpretation and testing	18
	Project output screenshots	19–21
6	CONCLUSION AND FUTURE WORK	22
7	REFERENCES	23

ABSTRACT

AI Rescue Mission is an interactive educational project developed for the course Foundations of Artificial Intelligence. It demonstrates how an intelligent agent finds a minimum-cost rescue route in a grid containing obstacles. The application combines a search simulator with a Player versus AI rescue race, allowing learners to compare human navigation with an autonomous drone driven by the actual A* algorithm.

The system represents each traversable grid cell as a state and permits four-direction movement with a unit step cost. A* evaluates states using f(n) = g(n) + h(n), where g(n) is the path cost accumulated from the start and h(n) is the Manhattan distance to the goal. Breadth-First Search provides an uninformed baseline on the identical map. Search traces, node scores, explored cells and the recovered path are derived from executed algorithms rather than predetermined output.

Across five predefined challenge maps, both algorithms produced identical optimal path costs. A* expanded 147 nodes in total compared with 566 for BFS, an aggregate reduction of 74.03%. The mean of individual map reductions was 73.85%. These results establish exploration savings for the selected maps, not a universal runtime guarantee. All search and game logic executes locally in the browser without machine learning, external AI APIs or a database. The project connects theoretical search concepts with observable decisions and gameplay.

CHAPTER 1

INTRODUCTION

Artificial Intelligence studies agents that perceive an environment and select actions to achieve a goal. Search is one of its fundamental problem-solving techniques: an agent examines possible states and transitions until it obtains a solution. In a rescue setting, the objective is to move from an initial location to a survivor while avoiding blocked terrain and minimizing travel cost.

A grid is a useful abstraction because positions, permitted moves and obstacles can be represented precisely. Each free cell is a state; moving up, down, left or right creates an edge to a neighboring state. The goal test checks whether the current coordinates match the distress beacon. A successful solution is a sequence of adjacent, obstacle-free cells connecting the start to the goal.

1.1 Problem statement

Classroom explanations of heuristic search often introduce equations without showing how they alter the order of exploration. Learners may confuse estimated distance with actual path cost, assume that A* follows a straight line, or fail to see why obstacles can force a detour. A practical demonstration must expose both the discovered states and the final route.

AI Rescue Mission addresses this difficulty through two linked experiences. The simulator supports terrain editing, stepping through A* and comparison with BFS. The rescue race makes the same search problem playable: a human rover competes with an autonomous drone following an A*-computed route. The Field Manual preserves the academic explanation alongside the game.

1.2 Motivation

The project uses rescue navigation as a learning metaphor, not as a real emergency-response system. Its value lies in making state-space search,

heuristic evaluation, parent tracking and optimality conditions visible and reproducible.

1.3 Objectives

The primary objective is to demonstrate authentic informed search in a form that students can inspect and interact with. The project aims to implement A* and BFS on the same grid model; visualize the open and explored states; calculate g(n), h(n) and f(n); reconstruct a valid minimum-cost route; and compare exploration effort under controlled conditions.

A second objective is to connect algorithmic cost with gameplay. The player's successful moves increase the step count and consume battery, while the drone advances along the A* route. A countdown, race timer, campaign sectors, custom maps and post-race analysis make the problem engaging without replacing the underlying algorithm.

1.4 Scope and assumptions

The simulator supports square grids from 10 × 10 to 15 × 15. Movement is orthogonal, every legal edge costs one, and obstacles are impassable. A map remains fixed during a search or race. The goal is a single beacon, and the agent has complete knowledge of the supplied grid. These assumptions make Manhattan distance a suitable heuristic.

The application performs its search and game computations in browser memory. It has no application database, user-account system, machine-learning model or external AI API. Challenge completion and campaign progress are session-only. The website framework may serve the page, but no remote service computes the rescue path.

1.5 Expected learning outcomes

After using the system, a learner should be able to formulate a search problem, distinguish informed from uninformed search, explain admissibility and consistency, interpret a search trace, calculate Manhattan distance and evaluate a pathfinding result. The

implementation also illustrates the difference between an algorithm’s logical behavior and the data structure used to implement its frontier.

CHAPTER 2

LITERATURE REVIEW

2.1 Hart, Nilsson and Raphael (1968)

Study: “A Formal Basis for the Heuristic Determination of Minimum Cost Paths.” This foundational work introduces the A* approach for finding minimum-cost paths using heuristic information. It combines the known cost of reaching a state with an estimate of the remaining cost and establishes the theoretical basis for informed graph search.

Key finding and project relevance: A* can direct exploration toward promising states while retaining optimality under appropriate heuristic conditions. AI Rescue Mission applies this principle through f(n) = g(n) + h(n), with parent tracking to recover the route. The project restricts movement and cost so that its selected heuristic satisfies the required conditions.

2.2 Russell and Norvig (2021)

Study: Artificial Intelligence: A Modern Approach, fourth edition. The textbook explains problem-solving agents, state spaces, uninformed and informed search, heuristic functions, A* and optimal solution finding. It provides a broad conceptual foundation for the course.

Key finding and project relevance: A search problem must define the initial state, available actions, transition model, goal test and path cost. These elements are explicitly represented by the grid, four legal directions, obstacle rules, beacon location and unit move cost. BFS is included as a baseline to show the effect of adding heuristic information.

These two sources supply the theoretical framework. The practical sources discussed next explain how the framework can be translated into executable pathfinding and visual teaching tools.

2.3 Amit Patel (2014): Introduction to A*

Study: “Introduction to the A* Algorithm,” Red Blob Games. The interactive article demonstrates BFS, related search strategies and heuristic-based pathfinding. Visual explanations show how the frontier changes when the search uses information about the goal.

Key finding: Interactive diagrams make exploration order and path recovery easier to understand. The project adopts this educational principle through colored open and explored states, editable maps and a live node inspector.

2.4 Amit Patel (2022): Implementation of A*

Study: “Implementation of A*,” Red Blob Games. The implementation guide discusses graph representation, path costs, priority queues, parent links and heuristic calculation.

Key finding: Correct pathfinding requires careful management of the frontier and best-known costs, not merely a visual line between endpoints. The project maintains scores and parent records and selects the lowest-priority state from its sorted open array. The year 2022 is the survey label supplied for this project; the online page is an evolving tutorial.

2.5 Amit Patel (2014): Flow field pathfinding

Study: “Flow Field Pathfinding for Tower Defense,” Red Blob Games. The article explores how search results can guide movement toward a common destination, including approaches useful when multiple agents share a goal.

Key finding: Search output can inform an agent’s movement. In this project, the drone follows one A*-computed route. A shared flow field is not implemented; the source instead motivates a possible extension for multiple rescuers. The Patel sources are technical tutorials rather than peer-reviewed research papers.

2.6 Research gap

Understanding heuristic search can be difficult through theory alone. This project addresses the learning challenge by providing an interactive grid-based rescue simulation that visualizes A* search, Manhattan distance, explored nodes and the final path.

The gap addressed is educational rather than a claim that a new pathfinding algorithm has been invented. A* and BFS are established algorithms. The contribution is their integration into a single experience in which a learner can change the state space, observe the search and then experience its cost through a playable race.

2.7 Relationship to the proposed system

The theoretical sources explain why a heuristic can preserve optimality. The practical sources explain frontier management, path recovery and visual reasoning. The proposed system combines these ideas using a shared grid model, authentic algorithm outputs, a node inspector and a post-race radar view.

A clear distinction is maintained between estimate and outcome: h(n) ignores obstacles and is therefore not a guaranteed remaining route length; g(n) is the cost of the discovered route to a node; and the completed path is reconstructed from parent links. A* may need to explore a detour even when it prioritizes states closer to the beacon.

2.8 Literature-based evaluation criteria

The report evaluates whether returned paths are legal, whether A* and BFS agree on the optimal cost for unit-cost maps, whether trace records reflect actual execution and whether fewer states are expanded on the selected challenges. The study does not measure student learning gains, compare teaching methods statistically or evaluate real disaster-response performance.

Full bibliographic entries for the five sources appear in Chapter 7 in alphabetical order of the first author. In-text references use the author–year form throughout this report.

CHAPTER 3

PROPOSED SYSTEM

The proposed system contains a browser-based simulation layer and a rescue-game layer that share the same A* search implementation. A grid snapshot contains the dimensions, start, goal and set of blocked coordinates. This snapshot is the input to pathfinding; visualization and gameplay consume the resulting data rather than generating independent routes.

3.0 System workflow

First, the learner selects a campaign sector, loads a challenge or edits a custom map. The system represents coordinates as x and y values and obstacles as a set of coordinate keys. Second, A* computes the route using unit edge costs and Manhattan distance. Third, the simulator reveals recorded search events, while the game advances the drone along the recovered path.

When comparison is requested, BFS receives the identical current grid, including the same dimensions, start, goal and obstacles. The output panels show path cost and explored-node counts. A race result displays the winner and compares the player's traveled moves with the A* optimal route cost.

Functional requirements

The system must permit valid grid edits, prevent movement through obstacles, calculate authentic A* and BFS results, detect failure when no route exists, display node scores, expose a trace and support keyboard and on-screen rover controls. It must separate setup, countdown, racing and completed states so that gameplay is tied to the current map.

Design requirements

The interface must remain readable on desktop and narrow screens, identify search states and offer animation and sound controls. Fixed inputs produce deterministic search results except duration. The Field Manual retains the academic explanation.

3.1 ARCHITECTURE DIAGRAM

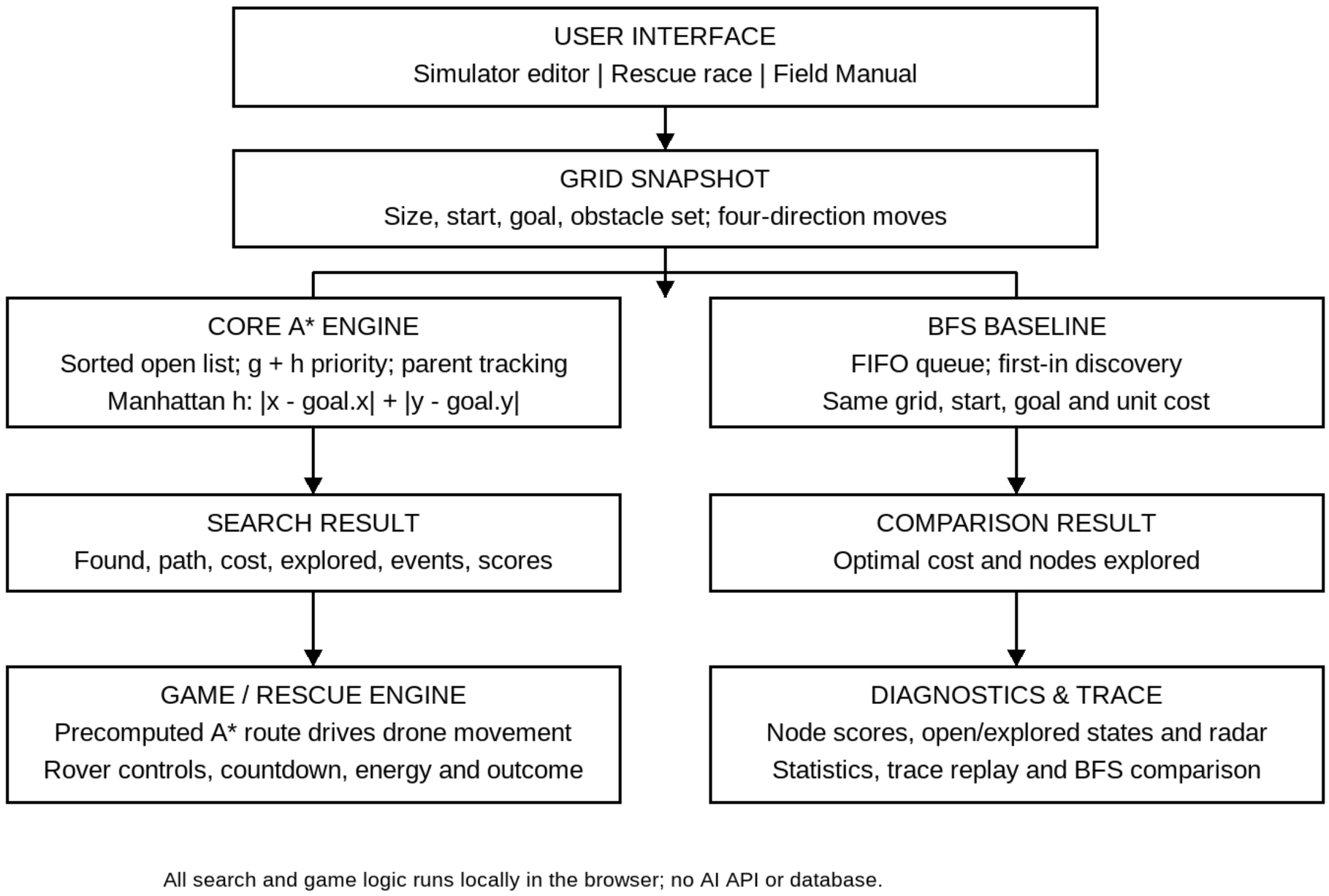


Figure 3.1: Browser-local architecture of AI Rescue Mission.

The Core A* Engine requests a Manhattan estimate while evaluating neighboring states. Its result supplies both the drone route and the search diagnostics. BFS is a separate baseline that receives the same grid snapshot. The diagram represents logical modules; the Manhattan calculation is a helper within the shared search code rather than a remote service.

The architecture does not contain a database, external AI model or pathfinding server. Search data remains in memory for the current visit. This separation allows the educational display and race mechanics to use the same verified path result.

CHAPTER 4

MODULES DESCRIPTION

4.1 Core A* Engine

Input: grid size, start coordinate, goal coordinate and obstacle set. Output: found status, ordered path, path cost, explored count, event count, duration, trace events and best-known scores. The implementation is written in TypeScript and executes as JavaScript in the application.

The engine starts with g(start) = 0 and adds the start to the frontier. At each iteration it sorts the open array by ascending f = g + h, then ascending h, then insertion order. This implements priority behavior with a sorted array, not a binary-heap priority queue. A closed set prevents a state from being expanded twice.

For each legal orthogonal neighbor, a tentative cost of current.g + 1 is calculated. If it improves the recorded score, the neighbor’s parent and cost are updated and the state is added to the open array. Reaching the goal triggers parent-chain reconstruction. If the frontier is exhausted, the result reports that no route was found.

Algorithm outline

1. Insert the start with g = 0 and h = Manhattan(start, goal). 2. Select the frontier entry with minimum (f, h, order). 3. Skip already-closed entries; close and record the current node. 4. If it is the goal, reconstruct and return the route. 5. Relax legal neighbors and record improved scores. 6. Repeat until success or frontier exhaustion.

Optimality and implementation trade-off

With unit orthogonal costs, Manhattan distance is consistent and A* returns a minimum-cost route. Sorting a frontier of k entries costs O(k log k) per sort and removing its first entry also incurs array movement. This is acceptable for small educational grids but should not be described as heap-based scaling.

4.2 Heuristic Manhattan Engine

The heuristic module estimates the remaining obstacle-free travel cost between a current cell and the goal. Its formula is h(n) = |x(n) − x(goal)| + |y(n) − y(goal)|. Absolute differences count the required horizontal and vertical coordinate changes.

For the Easy Rescue start (1, 8) and goal (8, 1), h(start) = |1 − 8| + |8 − 1| = 14. Initially g(start) = 0, so f(start) = 14. If a discovered node has g = 3 and h = 11, its priority is f = 14. Obstacles may increase the true route length but cannot reduce the number of coordinate changes required.

Admissibility and consistency

A heuristic is admissible if it never exceeds the true minimum remaining cost h*(n). In this grid, at least |Δx| horizontal moves and |Δy| vertical moves are necessary. Therefore h(n) ≤ h*(n). It is consistent because a legal unit-cost move changes Manhattan distance by at most one, giving h(n) ≤ 1 + h(neighbor). At the goal, h = 0.

These properties apply to the stated four-direction, unit-cost model. They must be reconsidered if diagonal movement, weighted terrain, teleports or other transition rules are added. The visual arrow in the inspector indicates the goal’s direction; it is not a collision-free navigation instruction.

Educational comparison

The Field Manual also calculates Euclidean and Chebyshev distances to compare geometric estimates. Euclidean distance is the straight-line distance; Chebyshev distance is the larger coordinate difference. The operational A* engine continues to use Manhattan distance. The comparison interface does not switch the production search to these alternative heuristics.

4.3 Game / Rescue Engine

The game module turns a route-planning exercise into a Player versus AI race. The human rescue rover accepts Arrow keys, WASD and on-screen directional controls. Boundary and obstacle checks reject invalid moves. Every successful rover move increments the step count and consumes one energy unit, including revisiting a previously occupied cell.

The state sequence is ready → countdown → racing → over. A fresh A* route is computed when the race begins. Three countdown stages occur at 750 ms intervals. Once racing begins, a timer records elapsed time and the drone advances along the precomputed optimal path. The game does not run a new search for every drone move.

Campaign and custom mode

The campaign contains Clear Streets, Collapsed Highway, Urban Ruins Maze, Downtown Gridlock and Mega Catastrophe. Their grids range from 10 × 10 to 15 × 15. Drone move delays decrease from 620 ms in the first sector to 300 ms in the fifth. Custom races allow obstacle editing and random-city generation; the custom drone delay is 420 ms per move.

Energy, outcome and analysis

The battery budget is max(8, round(1.6 × optimal cost) + 4). For an optimal cost of 14, the initial budget is 26 energy units. Reaching the beacon ends the race in the player's favor; drone arrival or exhausted battery produces the corresponding non-player outcome. Rematch and sector progression controls support repeated play.

The post-race view compares player movement with optimal route cost and can reveal AI radar states. A faster human finish does not prove a shorter route: human input rate and the drone's configured delay affect the winner. Browser-generated audio supplies move, countdown, start, win, lose and radar cues; a mute control disables them.

4.4 Diagnostics & Trace

This module translates algorithm output into inspectable evidence. Each A* event records the step number, node coordinate, g, h, f, parent, open-list size and whether the state was added to open or explored. The simulator replays the events at a selected speed or advances them one at a time.

The node inspector calculates Manhattan distance for the selected cell and shows g and f when a score has been discovered. The statistics panel reports path availability, route cost, explored states, open states, heuristic name and measured execution time. A collapsible table exposes the event sequence and parent links.

Trace semantics

A* records discovery/update events and expansion events. BFS records only expansion events. Consequently, raw trace-event totals are not identical units of computational work across the two algorithms. On Easy Rescue, A* produces 48 events but expands 15 nodes; BFS produces 88 events and expands 88 nodes. Explored-node counts are therefore the primary comparative metric in this report.

An open-list state is a discovered frontier candidate; an explored state has been selected for expansion. Frontier candidates can remain when the goal is reached. The completed route is recovered through parent records and highlighted separately from all examined states.

Radar and teaching support

The radar view reveals the search footprint and optimal path after a race, explaining the route the drone used. The Field Manual contains problem formulation, heuristic examples, pseudocode, applications and learning outcomes. Challenge completion costs are retained only for the current visit.

Measured duration uses a high-resolution timer around the search function. It does not include animation, player reaction time or the drone’s

movement delays. A displayed value rounded to 0.00 ms is not evidence that execution requires zero time.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 Experimental setup

The interface uses React, TypeScript, TanStack Start and Vite. Search uses arrays, maps and sets. Report data was obtained by executing the current A* and BFS functions on five deterministic simulator challenges.

For each map, both functions received identical dimensions, start, goal and blocked-cell coordinates. Movement was restricted to four directions and each legal edge had cost one. No obstacles were changed between the two executions. The reported path cost is the number of edges, equal to returned path length minus one.

Evaluation procedure

For each challenge: construct the obstacle set, run A*, run BFS, confirm success, compare path costs and record explored nodes. Per-map reduction is 100 × (BFS explored − A* explored) / BFS explored. Aggregate reduction uses summed node counts. The arithmetic mean averages the five percentage reductions; these are different summary measures.

Output verification

The application was opened in Chromium at a 1280-pixel-wide desktop viewport. The Field Manual was expanded, animation set to Instant, A* run to completion and the same-map BFS comparison requested. The interface displayed a 14-move route, 15 A* explored nodes and 88 BFS explored nodes, matching the independent execution. No page runtime error was observed during this captured flow.

These tests measure correctness and exploration, not controlled runtime, usability or real robotics performance. Absolute speed rankings are not claimed.

5.2 Results & comparison with BFS

Both algorithms found a valid route on every challenge and returned the same optimal cost. The cost column below applies to both A* and BFS; explored counts include the goal expansion.

Challenge	Cost	A* nodes	BFS nodes	Reduction
Easy Rescue	14	15	88	82.95%
Blocked Route	15	36	82	56.10%
Maze Rescue	22	23	113	79.65%
Multiple Routes	17	44	121	63.64%
Expert Rescue	28	29	162	82.10%
Total	96	147	566	74.03%

Table 5.1: Measured results on the five simulator challenges.

A* expanded 419 fewer nodes across the five maps. Aggregate reduction = (566 − 147) / 566 × 100 = 74.03%. The arithmetic mean of the five individual reductions is 73.85%. The total cost of 96 is simply the sum of five separate routes, not the length of a single mission.

Interpretation

The heuristic reduced exploration substantially in these scenarios while preserving optimal cost. Blocked Route and Multiple Routes required more A* expansions than nearly direct cases, illustrating that a distance estimate does not encode obstacle geometry. These observations apply to the tested maps; they do not establish that A* always explores fewer nodes for every possible grid or always executes faster than BFS.

5.3 Correctness checks and interpretation

Equal path cost is expected because BFS is optimal for unit-cost edges and A* uses an admissible, consistent heuristic in this movement model. A legal path must start at the specified start, end at the goal, remain within the grid, avoid blocked cells and have Manhattan distance one between consecutive cells.

A supplementary source-level smoke check used a reachable open grid, a blocked goal, start equal to goal and simple Manhattan calculations. These cases check ordinary success, frontier exhaustion, the zero-move solution and basic heuristic values. They do not constitute exhaustive formal verification.

Reported trace counts

Challenge	A* events	BFS events
Easy Rescue	48	88
Blocked Route	87	82
Maze Rescue	62	113
Multiple Routes	113	121
Expert Rescue	83	162

A* can produce more events than BFS even when it explores fewer nodes because its trace records both frontier insertion and expansion. Event count is a teaching diagnostic, not a fair stand-alone speed measure.

Limitations of the evaluation

Five designed maps are a limited sample and do not represent every obstacle distribution. Race outcomes depend on input timing and fixed drone speed. No claim of improved exam scores or learner performance is made. The screenshots on the following pages are actual application output captured during verification, rather than mock-ups or expected-result placeholders.

5.4 Project output: rescue race

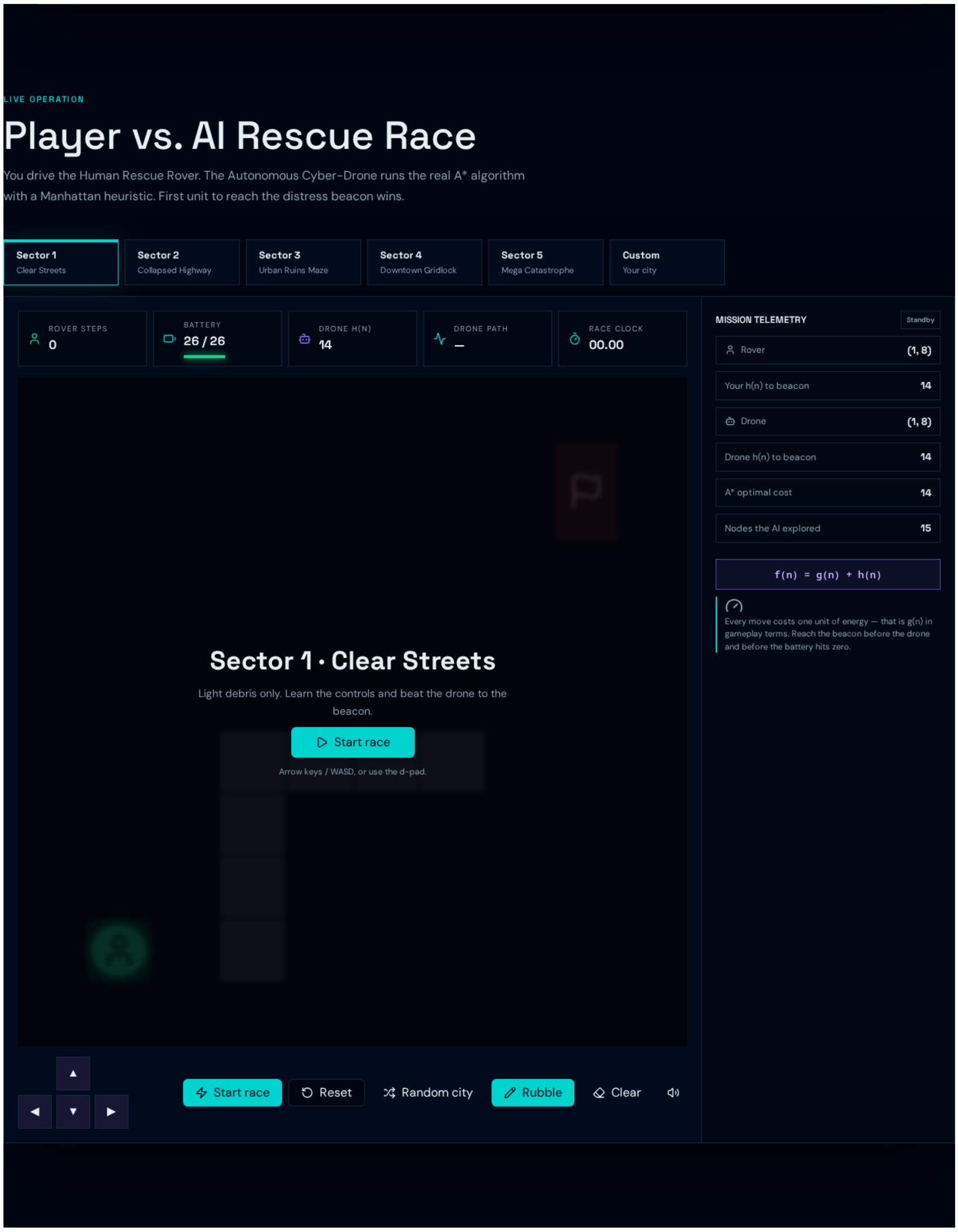


Figure 5.1: Campaign rescue race in the ready state.

The screenshot shows the selected campaign sector, player and drone HUD, map, mission controls and rover directional pad. Obstacles define the traversable state space, while the beacon identifies the goal. This ready-state output demonstrates the playable setup; it is not evidence of a completed human victory.

5.5 Project output: A* simulator

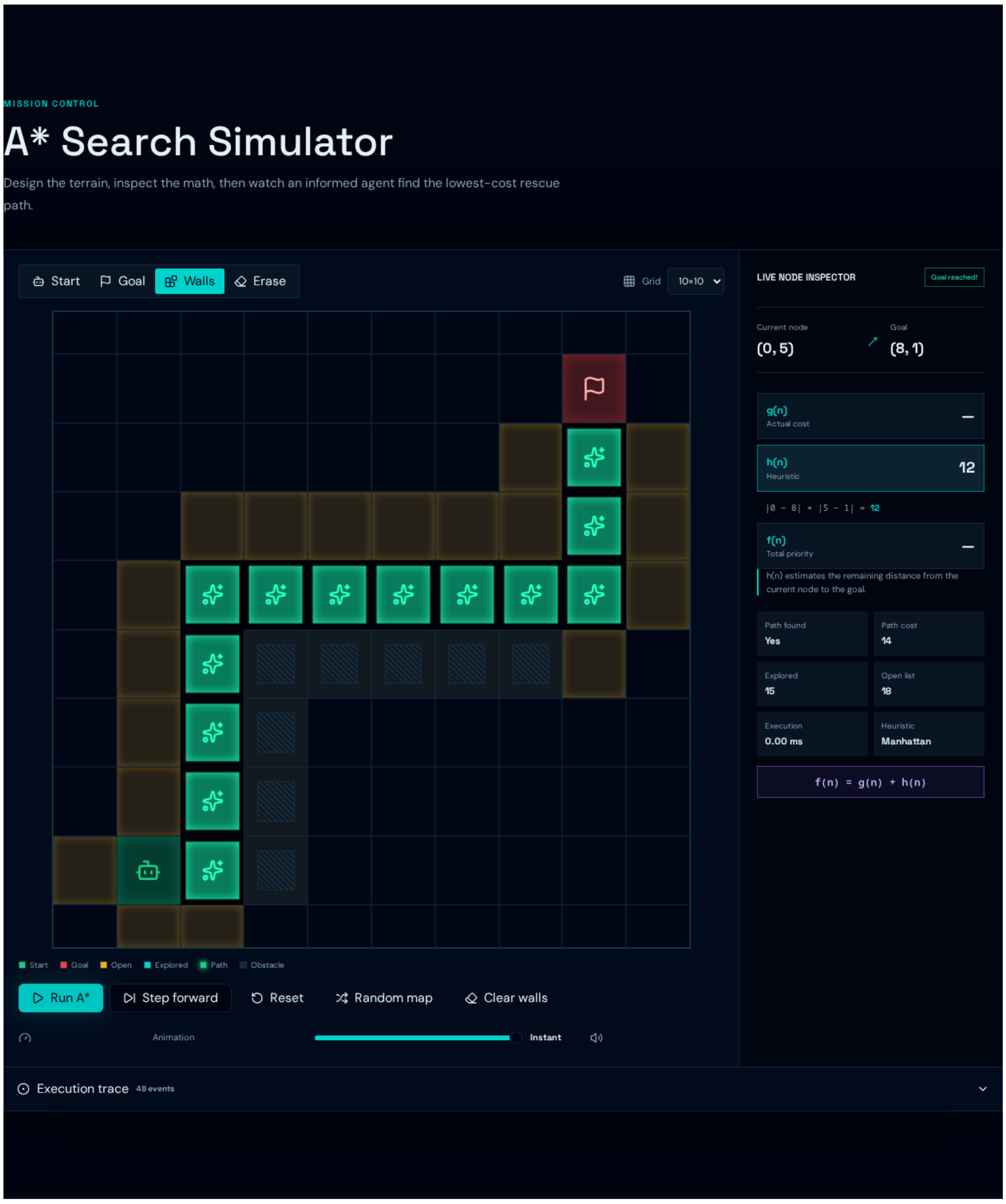


Figure 5.2: Completed A* search on the Easy Rescue map.

The displayed result reports Path found: Yes, Path cost: 14 and Explored: 15. The highlighted route contains the intermediate steps between the rover start and beacon. The trace contains 48 recorded events. The inspector computes h for the currently selected cell; an undiscovered cell can legitimately have no g or f value.

5.6 Project output: BFS comparison

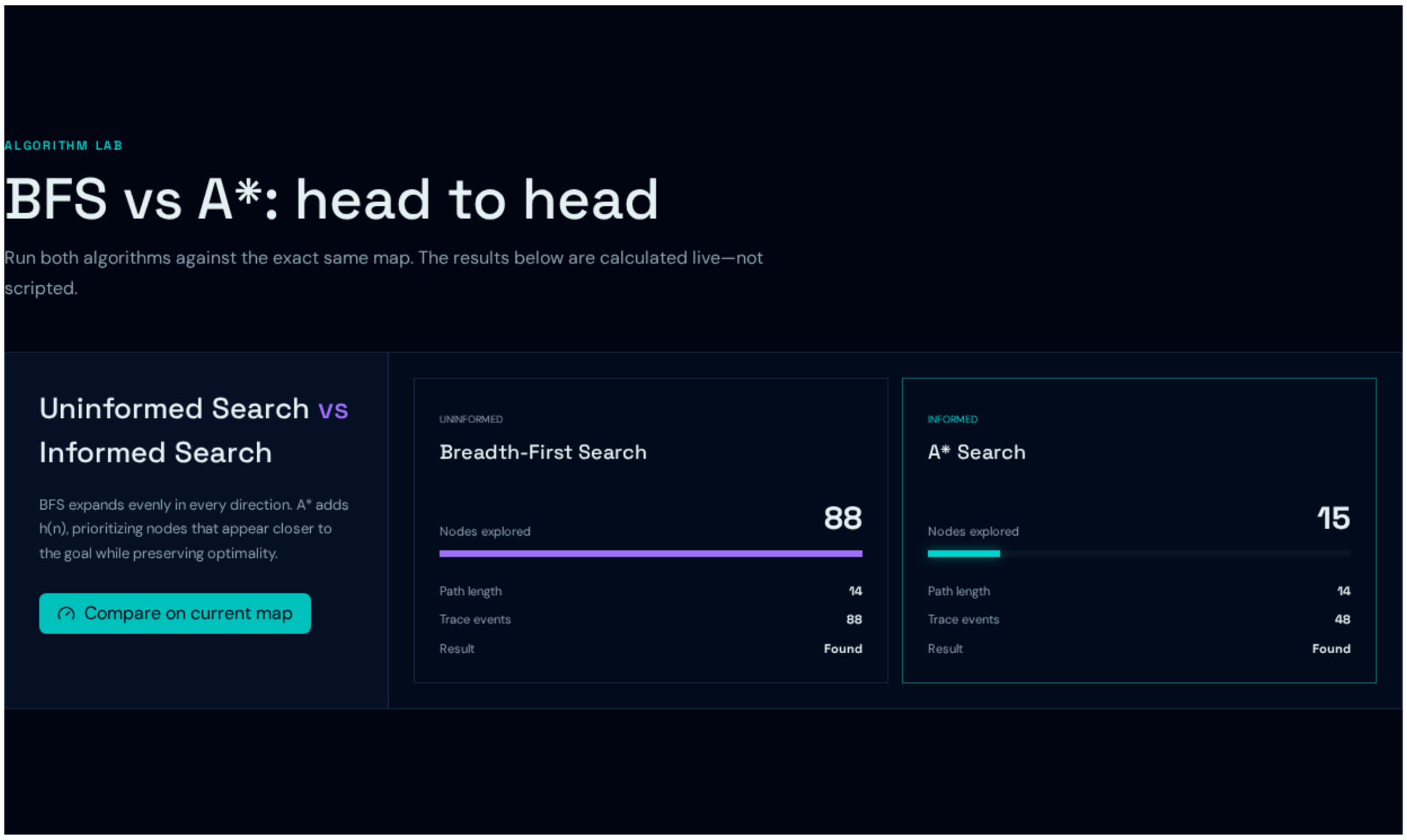


Figure 5.3: BFS and A* evaluated on the identical current grid.

The comparison panel shows BFS exploring 88 nodes and A* exploring 15 nodes, while both return a 14-move path. The visible trace totals are 88 for BFS and 48 for A*, consistent with their different logging policies. The panels are populated by real function execution.

Reproducing the output

Open the Field Manual and retain the default Easy Rescue challenge. Set animation to Instant and run A* until Goal reached appears. Select Compare on current map without editing the grid. The deterministic cost and explored counts should match the figures; execution duration may vary between devices and runs.

The screenshot does not establish performance on all maps by itself. Table 5.1 supplements the visual output with independently executed results from the five named challenges. Campaign-sector maps are separate from simulator-challenge maps and are not substituted into that table.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

AI Rescue Mission demonstrates heuristic search through a usable simulator and a playable rescue race. The project formulates a static navigation problem explicitly, computes routes with authentic A*, uses Manhattan distance for prioritization and exposes the resulting path and search decisions. BFS provides an understandable uninformed baseline.

Across five predefined challenges, A* and BFS produced identical optimal costs. A* explored 147 states compared with 566 for BFS, corresponding to a 74.03% aggregate reduction and a 73.85% mean per-map reduction. The project therefore demonstrates reduced exploration for the selected scenarios without claiming a new search algorithm or universal speed improvement.

6.2 Current limitations

The environment has unit-cost orthogonal movement, one destination and fixed obstacles. A* uses a sorted open array rather than a binary heap; BFS uses an array queue. Progress is session-only. Drone motion follows a completed route rather than online replanning. The model does not include sensor uncertainty, physical dynamics or emergency-service constraints.

6.3 Future work

Future versions could replace the sorted frontier with a binary heap and use an indexed FIFO queue for larger maps. Weighted terrain, diagonal movement or dynamic obstacles would require revised heuristics and correctness checks. Multiple rescuers could motivate flow fields or coordinated planning. These are proposed extensions, not features already implemented.

Future evaluation could assess learner understanding with consent, export traces and test accessibility. The present project is a self-contained illustration of foundational AI search concepts.

CHAPTER 7

REFERENCES

Hart, P. E., Nilsson, N. J. and Raphael, B. (1968) ‘A Formal Basis for the Heuristic Determination of Minimum Cost Paths’, IEEE Transactions on Systems Science and Cybernetics, Vol. 4, No. 2, pp. 100–107.

Patel, A. (2014a) ‘Flow Field Pathfinding for Tower Defense’, Red Blob Games. Available at: https://www.redblobgames.com/pathfinding/tower-defense/ (accessed 10 October 2026).

Patel, A. (2014b) ‘Introduction to the A* Algorithm’, Red Blob Games. Available at: https://www.redblobgames.com/pathfinding/a-star/introduction.html (accessed 10 October 2026).

Patel, A. (2022) ‘Implementation of A*’, Red Blob Games. Available at: https://www.redblobgames.com/pathfinding/a-star/implementation.html (accessed 10 October 2026). The survey year follows the supplied literature list; the online tutorial is maintained over time.

Russell, S. J. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach, 4th ed., Pearson.

Project implementation: JavaScript-based A* Search Algorithm using Manhattan Distance as the heuristic function.